



Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 5

Belo Horizonte
7 de junho de 2026

SUMÁRIO

1	EXERCÍCIO 9.2	3
	ANEXO A CÓDIGO EXERCÍCIO 9.2	6

1 EXERCÍCIO 9.2

Todo o código usado nesse exercício encontra-se no Anexo A.

Considerando o problema do exercício 8.2 em que temos um domínio 2D quadrado e queremos simular o modo TM_z com o FDTD, usamos os seguintes parâmetros de discretização:

- $N_x = 200$, $N_y = 200$
- $\Delta y = \Delta x = 1$ mm
- $\Delta t = \frac{0.9}{c\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 2.12$ ps (condição CFL 2D)
- $N_t = 800$

A fonte aplicada no centro do domínio é uma função gaussiana dada por

$$f(t) = e^{-\frac{1}{2}\left(\frac{t-40}{12}\right)^2}$$

que tem um pico em $t = 84$ ps e uma largura de aproximadamente 25 ps.

Os parâmetros da CPML adotados foram os seguintes:

- $N_{pml} = 30$, $m = 3$
- $R_{err} = 10^{-6}$
- $\epsilon_{\eta} = \sqrt{\frac{\mu}{\epsilon}}$
- $\sigma_{max} = -\frac{(m+1)\ln(R_{err})}{2\eta d_{pml}} = 2.44$

A propagação obtida está exibida na Figura 1, onde podemos observar que o campo se propaga para fora do domínio sem reflexões significativas, indicando que os PMLs estão funcionando corretamente. Com uma ponta de prova próximo ao centro do grid, verificamos uma leve reflexão do campo, como mostra a Figura 2.

O coeficiente de reflexão pode ser calculado a partir dos picos verificados na Figura 2, obtendo:

$$R = \frac{0.00007}{0.01} = 0.007 = 0.7\%$$

o qual, quando comparado com a reflexão obtida usando o Mur de $R = 5\%$ no exercício 8.2, representa uma redução quase 10 vezes melhor usando a PML.

A Figura 3 mostra a evolução do campo E_z ao longo do tempo, em um corte horizontal no eixo x passando pelo centro da cavidade. As linhas verticais vermelhas indicam a região onde o campo é absorvido pelos PMLs, e podemos observar que o campo decai para zero em todos os instantes, evidenciando a absorção da PML.

Alterando o ângulo θ da ponta de prova em relação ao centro do grid, podemos verificar como a reflexão da PML varia em relação ao ângulo de incidência. Variando

Figura 1 – Propagação de E_z ao longo do grid, sem reflexão nas bordas.

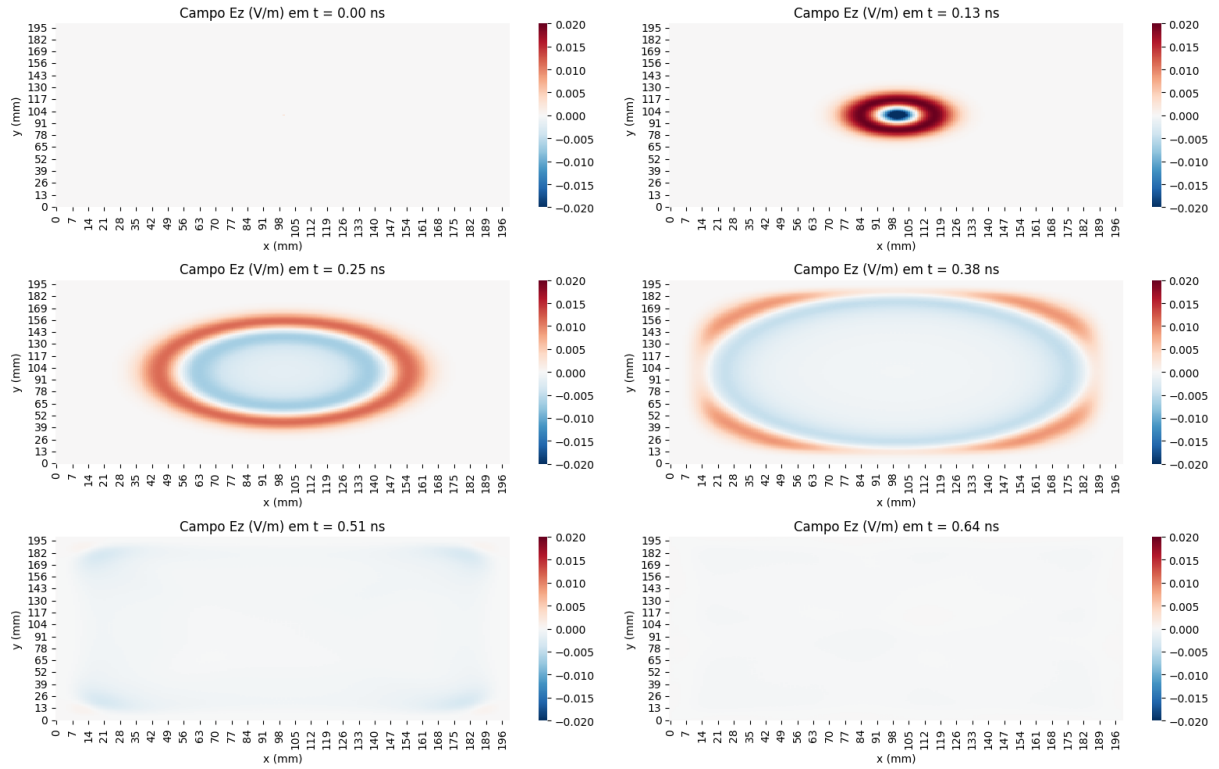


Figura 2 – Reflexão de E_z no centro do grid em função do tempo.

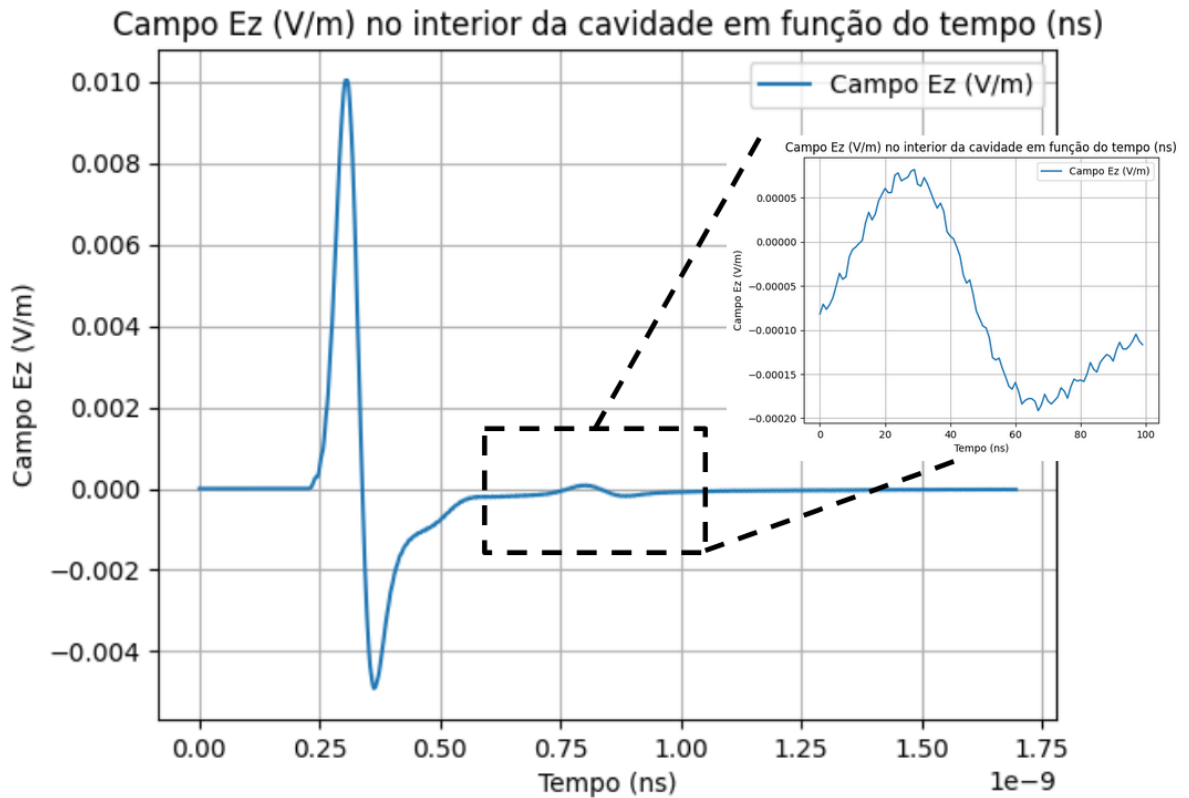
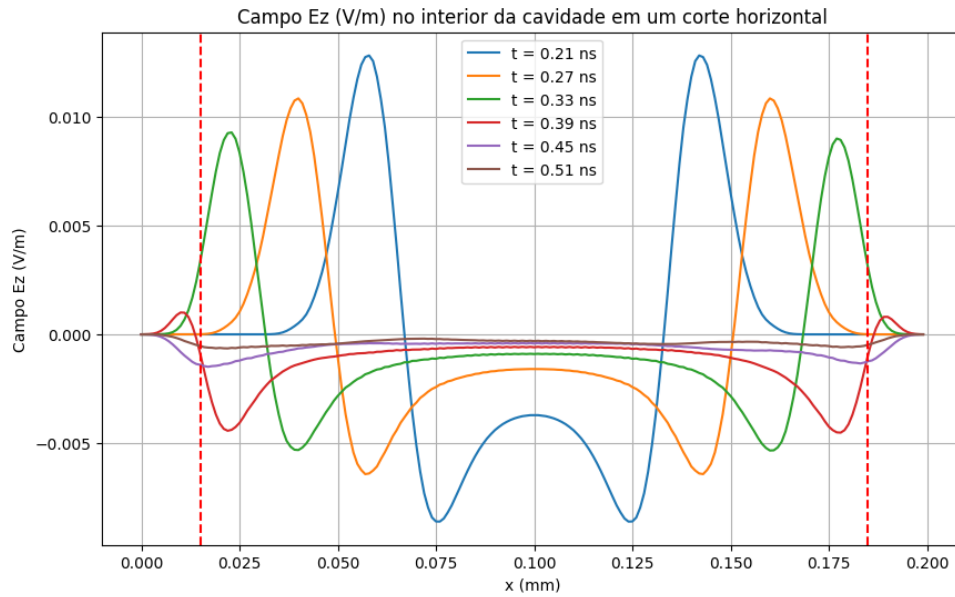


Figura 3 – Decaimento do campo E_z ao longo do eixo x ao entrar nas regiões de PML.



as posições p_x e p_y da ponta de prova, tendo em mente que o ponto $(N_x/2, N_y/2)$ corresponde ao centro do grid, obtemos os seguintes resultados para o coeficiente de reflexão R , exibidos na Tabela 1:

Tabela 1 – Variação do coeficiente de reflexão com o ângulo de incidência.

p_x	p_y	θ	R
$N_x/2 + 0$	$N_y/2 + 50$	0°	0.013
$N_x/2 + 50$	$N_y/2 + 30$	30°	0.016
$N_x/2 + 50$	$N_y/2 + 50$	45°	0.019
$N_x/2 + 30$	$N_y/2 + 50$	60°	0.016
$N_x/2 + 50$	$N_y/2 + 0$	90°	0.013

Vemos que a reflexão é máxima quando a onda incidente está inclinada em 45 graus em relação à normal da borda, e é mínima quando a onda incide perpendicularmente (0 ou 90 graus). Assim, temos um desempenho pior da PML nas quinas do domínio, conforme esperado.

ANEXO A – CÓDIGO EXERCÍCIO 9.2

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
min_lambda = vp / 1e9 # comprimento de onda mínimo para 1 GHz = 3e-1 m
Nx = Ny = 200
dx = dy = 0.1 * min_lambda
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 250
L = Nx * dx
T = Nt * dt

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(Nt // 2, Nt - 5, 6, dtype=int)
Ez_snapshots = []

cx, cy = Nx // 2, Ny // 2
px, py = cx, cy # pontas de prova

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

grid_x_Hx = np.linspace(0, L, Nx + 1)
grid_y_Hx = np.linspace(dy/2, L - dy/2, Ny)
grid_t_Hx = np.linspace(dt/2, T - dt/2, Nt)

grid_x_Hy = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hy = np.linspace(0, L, Ny + 1)
grid_t_Hy = np.linspace(dt/2, T - dt/2, Nt)

# --- campos ---
Ez_curr = Ez_next = np.zeros((grid_x_Ez.shape[0], grid_y_Ez.shape[0]))
Hx_curr = Hx_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0]))
Hy_curr = Hy_next = np.zeros((grid_x_Hy.shape[0], grid_y_Hy.shape[0]))

# ---- parâmetros da PML ----
Npml = 60
m = 3 # ordem polinomial
R0 = 1e-8 # reflexão alvo
sigma_max_x = -1 * (m + 1) * np.log(R0) / (2 * np.sqrt(mu / eps) * Npml * dx)
sigma_max_y = -1 * (m + 1) * np.log(R0) / (2 * np.sqrt(mu / eps) * Npml * dy)

# ---- vetores para salvar os sigmas ----
# sigma = 0 dentro da regioao, diferente de zero na PML
sigma_x = np.zeros(Nx + 1)
sigma_y = np.zeros(Ny)

for i in range(Npml):
    x = (Npml - i) / Npml
    sigma_x[i] = sigma_max_x * x**m

```

```

sigma_x[Nx - i] = sigma_max_x * x**m

for j in range(Npml):
    y = (Npml - j) / Npml
    sigma_y[j] = sigma_max_y * y**m
    sigma_y[Ny - j - 1] = sigma_max_y * y**m

# buffer de integração
l_curr = l_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0]))

# ponta de prova para coletar pontos para a FFT no centro da cavidade
probe = []

# marcha no tempo
for n in range(Nt-1):
    # calcula os coeficientes
    F1_x = ((2 * eps - dt * sigma_x[:]) / (2 * eps + dt * sigma_x[:]))
    F2_x = (1 / mu) * ( (2 * eps * dt) / (2 * eps + dt * sigma_x[:]) )

    F1_y = ((2 * eps - dt * sigma_y[:]) / (2 * eps + dt * sigma_y[:]))
    F2_y = (1 / mu) * ( (2 * eps * dt) / (2 * eps + dt * sigma_y[:]) )

    sigma_y_appended = np.append(sigma_y, [sigma_y[-1]])
    sigma_e = (sigma_x[1:-1][:, None] + sigma_y_appended[1:-1][None, :]) / 2
    G1 = (2 * eps - dt * sigma_e) / (2 * eps + dt * sigma_e)
    G2 = 2 * dt / (2 * eps + dt * sigma_e)

    Hy_next[:, :] = F1_x * Hy_curr[:, :] + F2_x * (
        (Ez_curr[1:, :] - Ez_curr[:, -1]) / dx
    )

    Hx_next[:, :] = F1_y * Hx_curr[:, :] - F2_y * (
        Ez_curr[:, 1:] - Ez_curr[:, -1]
    ) / dy + (sigma_y[:] * (dt ** 2) / (mu * eps)) * l_curr[:, :]

    # Hx_next[:, :] = Hx_curr[:, :] - (dt/(mu*dy)) * (
    #     Ez_curr[:, 1:] - Ez_curr[:, -1]
    # ) + (sigma_x[1:] * (dt ** 2) / (mu * eps)) * l_curr[:, :]

    Ez_next[1:-1, 1:-1] = G1 * Ez_curr[1:-1, 1:-1] + G2 * (
        (Hy_next[1:, 1:-1] - Hy_next[:, -1]) / dx
        - (Hx_next[1:-1, 1:] - Hx_next[1:-1, :, -1]) / dy
    )

    # Ez_next[1:-1, 1:-1] = G1 * Ez_curr[1:-1, 1:-1] + G2 * (
    #     (Hy_next[1:, 1:-1] - Hy_next[:, -1]) / dx -
    #     (Hx_next[1:-1, 1:] - Hx_next[1:-1, :, -1]) / dy
    # )

    l_next[:, :] = l_curr[:, :] + (Ez_curr[:, 1:] - Ez_curr[:, -1])

# --- fontes ---
frequency = 1e9
source_gaussiano = np.exp(-((n * dt - 1.8e-9)**2) / (0.6e-9)**2)
source_senoidal = np.sin(2 * np.pi * frequency * n * dt)
Ez_next[cx, cy] = source_gaussiano

# PEC nas bordas externas do grid
Ez_next[0, :] = Ez_next[Nx, :] = Ez_next[:, 0] = Ez_next[:, Ny] = 0

```

```
# salva o valor no centro para a FFT
probe.append(Ez_next[px, py])

# atualiza os campos para o próximo passo
Ez_curr, Ez_next = Ez_next, Ez_curr
Hx_curr, Hx_next = Hx_next, Hx_curr
Hy_curr, Hy_next = Hy_next, Hy_curr
I_curr, I_next = I_next, I_curr

# salva para os snapshots
if n in n_snapshots:
    Ez_snapshots.append(Ez_curr.copy())
```